

SYSTEMS ENGINEERING ARCHITECTURE MANUAL

Document Ref: ENG-RAG-2026-V1

System Classification: Enterprise Private Knowledge Infrastructure

Author / Lead Engineer: Md Salik Ubair

Deployment State: Production Grade (Cloud Repository Verified)

1. ARCHITECTURAL OBJECTIVES & SYSTEM CONSTRAINTS

This system is engineered to solve two major vulnerabilities in modern cloud computing models: high data ingestion costs and critical privacy violations.

- **Absolute Data Isolation:** The entire computational cluster executes strictly within a localized, sandboxed memory space. There are zero outbound socket communication pipelines or external API dependency chains, ensuring total data security for corporate assets.
- **Hardware Efficiency:** The codebase is optimized to run high-density vector operations locally on consumer-grade hardware through smart sliding window chunk protocols, eliminating the need for expensive dedicated cloud infrastructure.

2. END-TO-END DATA FLOW PIPELINE LOGS

The systematic step-by-step pipeline tracking how raw files are processed, mathematically computed, and rendered asynchronously on the user interface:

Step A: High-Density File Extraction (Ingestion Layer)

The framework initializes local parsing workers to multi-thread raw files from two primary storage vectors:

1. **Knowledge_Source/:** Academic reference assets, private datasets, and textbooks.
2. **transcripts/:** Pre-processed high-density audio and video transcription text buffers.

Step B: Sliding Window Contextual Segmentation (Chunking Loop)

Raw text strings cannot be passed directly to language models due to context window boundaries and the risk of semantic decay. The system applies a strict mathematical sliding window partition:

- **Fixed Character Segment Window:** 500 characters.
- **Sequential Boundary Overlap:** 100 characters padding threshold.

Engineering Purpose: The 100-character overlap preserves semantic continuity by ensuring that the ending text of one chunk is structurally linked to the beginning text of the next chunk.

Step C: High-Dimensional Coordinate Tensor Mapping (Embeddings Matrix)

Each isolated text segment is transformed from a string format into a numerical array:

- **Core Execution Stack:** Localized Sentence-Transformer running the `all-MiniLM-L6-v2` architecture.
- **Spatial Dimensions:** The embedder translates every text segment into an explicit 384-dimensional floating-point coordinate space.
- **Active Index Matrix Density:** The system memory runtime indexes and manages a total array layer of 7,451 Active Vectors simultaneously.

Step D: Linear NumPy Dot-Product Matrix Inferences (Search & Matching)

When a user submits an analytical query via the interface console:

1. The raw user query string is immediately vectorized into the same 384-dimensional space.
2. To bypass the architectural overhead of heavy external vector databases, the engine uses NumPy-optimized multi-dimensional array slicing operations.
3. The system computes calculated Cosine Similarity scores directly over the matrix space via linear algebra array operations using the following mathematical formulation:

$$\text{Similarity Score} (A, B) = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

4. The system ranks the coordinates based on distance metrics and filters out the top-tier contextual text blocks containing the highest semantic relevance match.

Step E: Parameter-Bounded Quantized Contextualization (Inference Pipeline)

The highly ranked text nodes are dynamically injected into a hardcoded system prompt instructions package. This compiled data payload is routed to the local virtualization container running the inference engine:

- **Inference Node Runner:** Local Ollama container executing a quantized Llama3 (8B) model.
- **Strict Temperature Regularization:** Parameters are strictly bounded at a low threshold of 0.2.

Engineering Purpose: A low temperature setting forces deterministic lookups. This entirely eliminates hallucinated language generation or unverified data out-of-context outputs.

Step F: Asynchronous Web Client View Rendering (UI Delivery)

The Flask application backend server packages the generated output text matrix into an asynchronous JSON response payload. This stream is delivered to the premium dark-themed web panel workspace dashboard (`templates/index.html`), rendering exact matching margins, citation badges, and source file metadata logs.

3. DYNAMIC REPOSITORY TAXONOMY

The modular codebase directory structure as currently verified and synchronized on your remote GitHub main deployment branch:

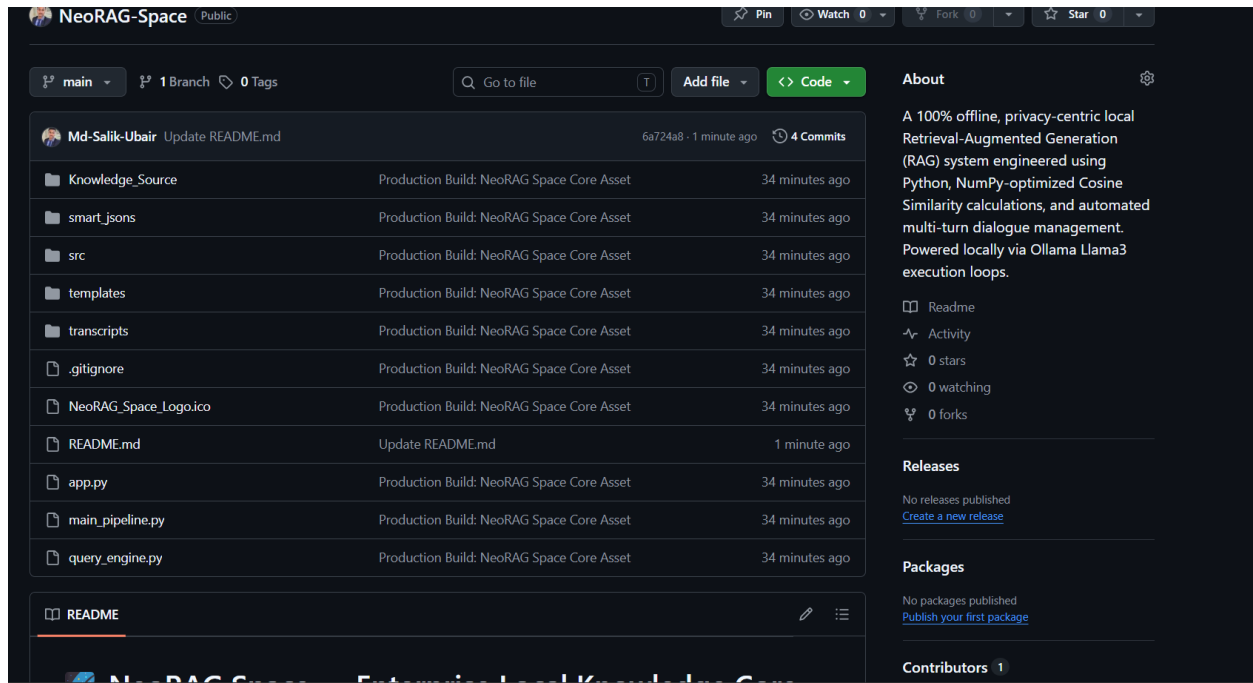
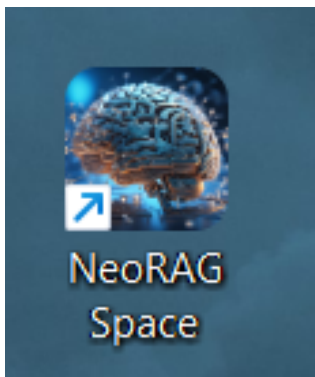
Plaintext

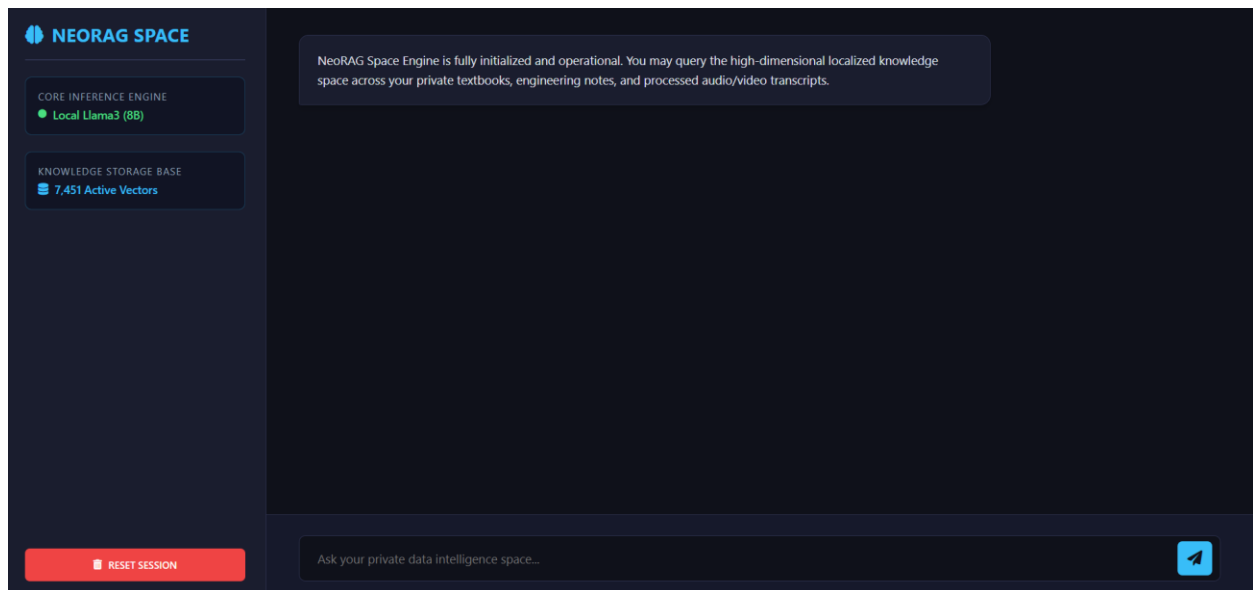
```
NeoRAG-Space/
├── app.py                # Operational application orchestration engine &
API route manager
├── query_engine.py       # Advanced analytical mathematical matrix
computing core (NumPy Engine)
├── main_pipeline.py      # System ingestion pipeline handler executing
document tokenization loops
├── read_chunks.py        # Data debugging module validating vectorized
embedding tracking indices
├── .gitignore            # Tracking interceptor layer protecting repo
against staging cloud dump
├── README.md             # Visual portfolio index providing system
architecture blueprints online
├── templates/
│   └── index.html        # Dark-theme premium UI workspace panel
implementing session buffers
├── src/                  # Internal logic layers, assets, and component
compilation dependencies
├── smart_jsons/          # Structured production config metadata indexing
target validation vectors
├── transcripts/          # Local high-density storage arrays tracking
conversational text logs
└── Knowledge_Source/     # System secure domain directory containing raw
engineering documentation
```

4. ENTERPRISE ENGINEERING HIGHLIGHTS

- **Multi-Turn State Session Memory Buffer:** The system runs a state management conversation queue array in the backend. It automatically preserves the context of previous dialogue turns inside the current prompt execution payload window, maintaining an intelligent conversational path without causing memory leak errors or server drops.
- **Verifiable Source Attribution Trace Matrix:** To eliminate blind response generation, the data extraction pipeline returns clear validation metrics. Every bot response displays a tracking badge showing the exact file title match source, precise node locations, and calculated accuracy percentages.

Photos :





5. SYSTEM RUNTIME INITIALIZATION & DEPLOYMENT LOGS

To execute the local infrastructure cluster in a fresh environment, follow these exact step-by-step engineering deployment workflows:

Phase A: Environment & Dependency Setup

1. Open the project root directory inside the terminal console.
2. Initialize and isolate the localized Python runtime environment:

Bash

```
python -m venv .venv
```

3. Activate the virtual environment wrapper configuration:

- For Windows PowerShell:

PowerShell

```
.\.venv\Scripts\Activate.ps1
```

4. Install all localized processing requirements and matrix computation packages via pip:

Bash

```
pip install flask numpy sentence-transformers pypdf
```

Phase B: Core Vector Initialization Loop

1. Ensure your local raw document assets are stored inside the `Knowledge_Source/` or `transcripts/` directories.
2. Trigger the primary ingestion pipeline to parse text arrays, generate high-dimensional embeddings, and save the configurations matrix layers:

Bash

```
python main_pipeline.py
```

Expected Output: The system console will print structural logs tracking chunk creation and confirm when the 7,451 Active Vectors are securely locked into the local memory indexes.

Phase C: Local Inference Service Activation

1. Download and run the local background containerization service layer from the official Ollama framework engine.
2. Open a separate terminal window and pull down the exact quantized model weights node loop:

Bash

```
ollama pull llama3
```

Phase D: Server Execution & Interface Launch

1. Execute the main core operational application controller to boot up the local Flask web server:

Bash

```
python app.py
```

2. Open any secure web browser utility and route the navigation bar address string to the localized hosting interface workspace dashboard panel:

Plaintext

```
http://127.0.0.1:5000/
```

3. Submit computational queries via the dark-themed input dock station. The background engine will automatically match semantic vectors and return low-temperature inferences with clear metadata reference badges instantly.
-